

Geometric Clifford Algebra Networks

Vladimirov Eduard, group M05-304a

31 октября 2024 г.

1 Introduction

Geometric transformations such as rotations, reflections, and translations are fundamental operations in mathematics and physics, with wide-ranging applications in computer graphics, robotics, and machine learning. Accurately modeling these transformations is crucial for tasks that require an understanding of spatial relationships and geometric structures.

Traditional neural networks often struggle to inherently capture and preserve geometric properties present in data, leading to the development of specialized architectures that incorporate geometric principles. One such mathematical framework that elegantly handles geometric transformations is **Geometric Algebra (GA)**. GA extends traditional vector algebra to higher dimensions and provides a unified language for representing geometric entities and operations.

Closely related to geometric algebra is **Clifford Algebra**, which generalizes complex numbers and quaternions to higher dimensions. Clifford algebras offer a powerful toolset for modeling rotations and reflections in multidimensional spaces, making them particularly useful for applications that involve complex geometric transformations.

This essay explores the fundamentals of geometric algebra and Clifford algebras, laying the groundwork for understanding how these mathematical structures can be integrated into neural network architectures, such as **Geometric Clifford Algebra Networks (GCANs)**. By leveraging the properties of GA and Clifford algebras, GCANs aim to inherently preserve geometric structures and improve the modeling of transformations within neural networks.

2 Fundamentals of Geometric Algebra

2.1 Definition and Basic Concepts

Let V be an n -dimensional vector space over a field F (typically $\mathbb{R}$) equipped with a non-degenerate symmetric bilinear form (inner product) $\langle \cdot, \cdot \rangle : V \times V \rightarrow F$.

Definition 2.1. The **Geometric Algebra** $\mathcal{G}(V)$ of the vector space V is the associative algebra over F generated by V with multiplication defined by the **geometric product**. The geometric product is a bilinear operation $\cdot : \mathcal{G}(V) \times \mathcal{G}(V) \rightarrow \mathcal{G}(V)$ satisfying:

1. **Associativity:**

$$(ab)c = a(bc), \quad \forall a, b, c \in \mathcal{G}(V).$$

2. **Distributivity over Addition:**

$$a(b + c) = ab + ac, \quad (a + b)c = ac + bc, \quad \forall a, b, c \in \mathcal{G}(V).$$

3. **Compatibility with Scalar Multiplication:**

$$(\alpha a)b = a(\alpha b) = \alpha(ab), \quad \forall \alpha \in F, \quad \forall a, b \in \mathcal{G}(V).$$

4. **Existence of Identity Element:** There exists an element $1 \in \mathcal{G}(V)$ such that $1a = a1 = a$, $\forall a \in \mathcal{G}(V)$.

5. **Defining Relation for Vectors:** For all $v \in V$,

$$v^2 = \langle v, v \rangle \cdot 1.$$

2.2 Geometric Product

The geometric product combines the inner and outer products, encapsulating both the metric and algebraic structure.

Definition 2.2. For vectors $u, v \in V$, the **geometric product** is defined as:

$$uv = u \cdot v + u \wedge v.$$

Properties. 1.

$$(uv)w = u(vw).$$

2.

$$uv \neq vu, \quad \text{in general.}$$

3.

$$u^2 = uu = u \cdot u + u \wedge u = u \cdot u,$$

4.

$$u \cdot v = \frac{1}{2}(uv + vu).$$

5.

$$u \wedge v = \frac{1}{2}(uv - vu).$$

2.2.1 Basis-Dependent Expression

Using an orthonormal basis $\{e_i\}$, the geometric product of basis vectors is:

$$e_i e_j = \begin{cases} 1, & \text{if } i = j, \\ e_i \wedge e_j, & \text{if } i \neq j. \end{cases}$$

For vectors $u = \sum_i u^i e_i$ and $v = \sum_j v^j e_j$, the geometric product expands to:

$$uv = \sum_i u^i v^i + \sum_{i < j} (u^i v^j - u^j v^i) e_i e_j.$$

2.3 Inner Product between Multivectors

Definition 2.3. For multivectors $A_r \in \bigwedge^r V$ and $B_s \in \bigwedge^s V$, the **inner product** (or **contracted product**) $A_r \cdot B_s$ is defined recursively using the properties of the geometric product and grade projection. One common definition is the **scalar product**:

$$A_r \cdot B_s = \langle A_r B_s \rangle_{|s-r|}, \quad \text{for } r, s \geq 0.$$

This definition ensures that the inner product between two multivectors yields a multivector of grade $|s - r|$.

Properties. 1. **Grade Reduction:** The inner product reduces the grade difference between multivectors.

2. **Symmetry:** For homogeneous multivectors,

$$A_r \cdot B_s = (-1)^{rs} B_s \cdot A_r.$$

3. **Associativity with Geometric Product:** The inner product is derived from the geometric product via grade projection.

2.4 Outer Product between Multivectors

Using basis blades, the outer product is computed by antisymmetrically combining the indices:

$$A_r \wedge B_s = \sum_{i_1 < \dots < i_r} \sum_{j_1 < \dots < j_s} A_{i_1 \dots i_r} B_{j_1 \dots j_s} e_{i_1} \wedge \dots \wedge e_{i_r} \wedge e_{j_1} \wedge \dots \wedge e_{j_s}.$$

Example 2.1. For $a = a^i e_i \in V$ and $B_2 = B^{jk} e_j \wedge e_k \in \bigwedge^2 V$, the outer product is:

$$a \wedge B_2 = \sum_i \sum_{j < k} a^i B^{jk} e_i \wedge e_j \wedge e_k.$$

This is a grade-3 multivector (trivector).

2.5 Multivectors and Grades

Definition 2.4. A **multivector** $A \in \mathcal{G}(V)$ is an element that can be expressed as a sum of r -vectors (also called grade- r elements):

$$A = \sum_{k=0}^n \langle A \rangle_k,$$

where $\langle A \rangle_k$ denotes the grade- k part of A , and $n = \dim V$.

Properties. 1. **Grade Projection:** The projection operator $\langle \cdot \rangle_k : \mathcal{G}(V) \rightarrow \bigwedge^k V$ extracts the grade- k component.

2. **Addition and Scalar Multiplication:** Multivectors form a vector space over F under addition and scalar multiplication.

3. **Multiplication of Multivectors:** The geometric product of multivectors of grades r and s results in multivectors containing grades from $|r - s|$ to $r + s$.

2.6 Examples

Example 2.2 (Complex Numbers). Let $V = \mathbb{R}$ with $Q(v) = v^2$. The Clifford algebra $\text{Cl}(\mathbb{R}, -v^2)$ is isomorphic to the field of complex numbers $\mathbb{C}$, where the imaginary unit i corresponds to a unit bivector satisfying $i^2 = -1$.

Example 2.3 (Quaternions). Let $V = \mathbb{R}^3$ with the quadratic form $Q(v) = -\|v\|^2$. The Clifford algebra $\text{Cl}(\mathbb{R}^3, Q)$ is isomorphic to the quaternions $\mathbb{H}$, with basis elements satisfying $e_i^2 = -1$ and $e_i e_j = -e_j e_i$ for $i \neq j$.

3 Pin and Spin Groups in Geometric Algebra

3.1 Definition of Pin Groups

Let V be a finite-dimensional vector space over a field F equipped with a non-degenerate quadratic form $Q : V \rightarrow F$. The Clifford algebra $\text{Cl}(V, Q)$ is constructed as previously defined, with elements called **multivectors** and multiplication given by the **geometric product**.

3.1.1 Construction of the Pin Group

The **Pin group** $\text{Pin}(V, Q)$ is a subgroup of the group of invertible elements in the Clifford algebra $\text{Cl}(V, Q)$. It is defined as follows:

$$\text{Pin}(V, Q) = \{u = v_1 v_2 \cdots v_k \mid v_i \in V, Q(v_i) = \pm 1, k \in \mathbb{N}\}$$

Here, each v_i is a unit vector in V with respect to the quadratic form Q , satisfying $v_i^2 = Q(v_i) = \pm 1$.

3.1.2 Properties of the Pin Group

1. **Adjoint Action on V :** The Pin group acts on V via the **adjoint action**:

$$v \mapsto uvu^{-1}, \quad \forall v \in V, u \in \text{Pin}(V, Q) \quad (1)$$

2. **Preservation of the Quadratic Form:** The adjoint action preserves the quadratic form Q :

$$Q(uvu^{-1}) = Q(v) \quad (2)$$

This follows from the properties of the geometric product and the definition of the Clifford algebra.

3. Double Cover of the Orthogonal Group: There exists a surjective group homomorphism:

$$\Theta : \text{Pin}(V, Q) \rightarrow \text{O}(V, Q)$$

defined by mapping $u \in \text{Pin}(V, Q)$ to the orthogonal transformation R_u on V given by $v \mapsto uvu^{-1}$. The kernel of Θ is $\{\pm 1\}$, so $\text{Pin}(V, Q)$ is a double cover of the orthogonal group $\text{O}(V, Q)$.

3.1.3 Example

Let $V = \mathbb{R}^n$ with the standard Euclidean quadratic form $Q(v) = v \cdot v$, where $v \cdot v$ is the usual dot product. The Clifford algebra $\text{Cl}_{n,0}(\mathbb{R})$ is generated by the orthonormal basis $\{e_1, e_2, \dots, e_n\}$ satisfying:

$$e_i e_j + e_j e_i = 2\delta_{ij}, \quad e_i^2 = 1$$

An element $u \in \text{Pin}(n)$ is a product of unit vectors e_i . For example, reflections are represented by $u = e_i$, and rotations can be represented by products of two unit vectors.

3.2 Definition of Spin Groups

3.2.1 Construction of the Spin Group

The **Spin group** $\text{Spin}(V, Q)$ is the subgroup of $\text{Pin}(V, Q)$ consisting of products of an even number of unit vectors:

$$\text{Spin}(V, Q) = \{a = v_1 v_2 \cdots v_{2k} \mid v_i \in V, Q(v_i) = \pm 1, k \in \mathbb{N}\}$$

3.2.2 Properties of the Spin Group

- 1. Adjoint Action on V :** The Spin group acts on V via the adjoint action, similar to the Pin group:

$$v \mapsto ava^{-1}, \quad \forall v \in V, a \in \text{Spin}(V, Q) \quad (3)$$

2. **Preservation of the Quadratic Form:** This action also preserves the quadratic form Q :

$$Q(ava^{-1}) = Q(v)$$

3. **Double Cover of the Special Orthogonal Group:** There exists a surjective group homomorphism:

$$\Theta : \text{Spin}(V, Q) \rightarrow \text{SO}(V, Q)$$

where $\text{SO}(V, Q)$ is the special orthogonal group (rotations preserving orientation). The kernel of Θ is $\{\pm 1\}$, so $\text{Spin}(V, Q)$ is a double cover of $\text{SO}(V, Q)$.

3.2.3 Example: Spin Group in Three Dimensions

Consider $V = \mathbb{R}^3$ with the Euclidean metric. The Clifford algebra $\text{Cl}_{3,0}(\mathbb{R})$ is generated by basis vectors e_1, e_2, e_3 with $e_i^2 = 1$ and $e_i e_j = -e_j e_i$ for $i \neq j$.

Elements of $\text{Spin}(3)$ are products of an even number of unit vectors. The Spin group $\text{Spin}(3)$ is isomorphic to the special unitary group $\text{SU}(2)$. The mapping Θ sends elements of $\text{Spin}(3)$ to rotations in $\text{SO}(3)$.

3.3 Geometric Product and Group Actions

3.3.1 Geometric Product in Clifford Algebras

The **geometric product** is the fundamental multiplication operation in Clifford algebras. For vectors $a, b \in V$:

$$ab = a \cdot b + a \wedge b \quad (4)$$

where:

- $a \cdot b = \frac{1}{2}(ab + ba)$ is the symmetric part, yielding a scalar.
- $a \wedge b = \frac{1}{2}(ab - ba)$ is the antisymmetric part, yielding a bivector.

This product is associative and encodes both the inner and outer products of vectors.

3.3.2 Adjoint Action via Geometric Product

The action of the Pin and Spin groups on V utilizes the geometric product. For $u \in \text{Pin}(V, Q)$ and $v \in V$, the adjoint action is given by:

$$v' = uvu^{-1}$$

Since u is composed of vectors and u^{-1} exists in $\text{Cl}(V, Q)$, this operation involves geometric products of vectors and their inverses.

3.4 Summary of Pin and Spin Groups

- The **Pin group** $\text{Pin}(V, Q)$ consists of products of unit vectors and represents all orthogonal transformations (rotations and reflections) of V .
- The **Spin group** $\text{Spin}(V, Q)$ consists of products of an even number of unit vectors and represents all proper rotations of V .
- Both groups act on V via the adjoint action using the geometric product, preserving the quadratic form Q .
- The Pin and Spin groups are double covers of the orthogonal group $\text{O}(V, Q)$ and the special orthogonal group $\text{SO}(V, Q)$, respectively.

4 Geometric Clifford Algebra Networks (GCANs)

4.1 Overview of GCANs and Their Objectives

- **Geometric Clifford Algebra Networks (GCANs):**
 - Neural network architectures integrating geometric algebra into computations.
- **Objective:**
 - Model and preserve geometric transformations within neural computations.
- **Approach:**

- Embed data and transformations using geometric algebra.
- Perform operations respecting geometric relationships (e.g., rotations, translations).
- Handle data with rich geometric structures more effectively than traditional neural networks.

4.2 Group Action Layers in GCANs

4.2.1 Definition and Mathematical Formulation

- **Group Action Layer:**

- Incorporates group actions from groups like $\text{Spin}(p, q, r)$ into layer computations.

- **Given:**

- Group G .
- Vector space X .
- Group action $\alpha : G \times X \rightarrow X$.

- **Layer Definition:**

$$T_{g,w}(x) = \sum_{i=1}^c w_i \cdot \alpha(g_i, x_i)$$

where:

- $x = (x_1, x_2, \dots, x_c) \in X^c$: Input elements.
- $g = (g_1, g_2, \dots, g_c) \in G^c$: Group elements.
- $w = (w_1, w_2, \dots, w_c) \in \mathbb{R}^c$: Scalar weights.
- c : Number of input channels.

- **Group Action in Geometric Algebra:**

- Using the sandwich product for rotations:

$$\alpha(g_i, x_i) = g_i x_i g_i^{-1}$$

where $g_i \in \text{Spin}(p, q, r)$ and $x_i \in \mathcal{G}_{p,q,r}$.

4.2.2 Preservation of Geometric Structure

- **Key Property:**

- Transformation $\alpha(g_i, x_i)$ maintains geometric properties of x_i .

- **Example:**

- If x_i is a vector (grade 1), then $\alpha(g_i, x_i)$ is also a vector.

- **Implications:**

- Network computations respect the geometric nature of data.
- Naturally model rotational symmetries and invariances.

4.3 Geometric Templates in Neural Networks

4.3.1 Concept and Benefits

- **Geometric Templates:**

- Predefined geometric transformations guiding learning and computation.

- **Implementation in GCANs:**

- Parameterize group actions (rotations, reflections) within network layers.

- **Benefits:**

- **Leverage Symmetries:** Utilize inherent data symmetries for improved learning.
- **Reduce Parameter Complexity:** Achieve complex transformations with fewer parameters.
- **Enhance Interpretability:** Understandable geometric transformations within the network.

4.3.2 Implementation in GCANs

- **Parameterization of Group Elements:**

- Learn group elements g_i during training.

- **Example:**

- Use geometric algebra $\mathcal{G}_{3,0,0}$ for rotations in $\mathbb{R}^3$.
- Parameterize rotors $R_i \in \text{Spin}(3)$ using bivector components.
- Rotor representation:

$$R_i = \exp \left(-\frac{1}{2}\theta_i B_i \right)$$

where:

- * θ_i : Rotation angle (learnable).
- * B_i : Unit bivector representing rotation plane (learnable).

4.4 Algorithmic Implementation

4.4.1 Pseudocode for a GCAN Layer

- **Group Action Layer Computation:**

Algorithm 1 Group Action Layer Computation

Require: Input multivectors $x_i \in \mathcal{G}_{p,q,r}$ for $i = 1, \dots, c$

Require: Learnable weights $w_i \in \mathbb{R}$ for $i = 1, \dots, c$

Require: Learnable rotors $R_i \in \text{Spin}(p,q,r)$ for $i = 1, \dots, c$

Ensure: Output multivector $y \in \mathcal{G}_{p,q,r}$

- 1: Initialize $y \leftarrow 0$
 - 2: **for** $i = 1$ to c **do**
 - 3: Transform input: $x'_i \leftarrow R_i x_i R_i^{-1}$
 - 4: Accumulate: $y \leftarrow y + w_i x'_i$
 - 5: **end for**
 - 6: **return** y
-

4.4.2 Parameterization of Rotors

- **Rotor R_i Parameterization:**

- Represented using exponential of bivectors:

$$R_i = \exp \left(-\frac{1}{2}\theta_i B_i \right)$$

where:

- * $\theta_i \in \mathbb{R}$: Rotation angle.
- * $B_i \in \mathcal{G}_{p,q,r}$: Unit bivector.

- **Bivector B_i :**

- Parameterized using basis bivectors:

$$B_i = b_i^1 e_1 e_2 + b_i^2 e_2 e_3 + b_i^3 e_3 e_1$$

where b_i^k are learnable parameters constrained such that $\|B_i\| = 1$.

4.4.3 GCA nonlinearity and normalization

We are interested in nonlinearities that support the idea of geometric templates. Therefore, we propose the following *Multivector Sigmoid Linear Unit* that gates a k -vector by applying

$$[\mathbf{x}]_k \mapsto \text{MSiLU}_k(\mathbf{x}) := \sigma(f_k(\mathbf{x})) \cdot [\mathbf{x}]_k,$$

where $f_k(\mathbf{x}) : G_{p,q,r} \rightarrow \mathbb{R}$ and σ is the logistic function. We choose this definition to ensure the geometric template: by only scaling, a k -vector remains a k -vector. In this work, we restrict f_k to be a linear combination of the multivector components, i.e.,

$$f_k(\mathbf{x}) := \sum_{i=1}^{2^n} \beta_{k,i} \cdot x_i,$$

where x_i denotes the i -th blade component of $\mathbf{x}$. Also, $\beta_{k,i}$ are either free scalar parameters or fixed to $\beta_{k,i} := 1$ or $\beta_{k,i} := 1/m$, resulting in summation or averaging, respectively. Similarly, we normalize a k -vector by applying a modified version of *group normalization*, where

$$[\mathbf{x}]_k \mapsto s_k \frac{[\mathbf{x}]_k - \mathbb{E}[[\mathbf{x}]_k]}{\mathbb{E}[\|[\mathbf{x}]_k\|]}.$$

The empirical average of $[\mathbf{x}]_k$ is computed over the number of channels specified by the group size hyperparameter, and rescaled using a learnable scalar s_k through $[\mathbf{x}]_k \mapsto s_k \cdot [\mathbf{x}]_k$, which again only scales the k -vector.

5 Experiments and Applications

5.1 Summary of Key Experiments from the GCANs Paper

To demonstrate the effectiveness of Geometric Clifford Algebra Networks, several experiments were conducted, showcasing the ability of GCANs to model complex geometric transformations and outperform traditional neural network architectures.

5.1.1 Modeling Rigid Body Transformations: The Tetris Experiment

One of the primary experiments involved predicting the motion of Tetris-like shapes undergoing rigid body transformations, including rotations and translations.

- **Objective:** Given a sequence of past positions and orientations of Tetris shapes, predict their future positions and orientations.
- **Dataset:** Simulated trajectories of Tetris shapes, each composed of multiple connected blocks, moving in three-dimensional space.
- **Challenges:**
 - Capturing both rotational and translational dynamics.
 - Maintaining the structural integrity of the shapes during prediction.
 - Dealing with complex interactions between connected components.

Data generation. We take the shapes as provided by **Thomas2018** and center them at the origin. For every Tetris shape, we randomly sample a rotation axis with a maximum angle of $0.05 \cdot 2\pi$ and translation directions with a maximum offset of 0.5. This leads to a rotation matrix P and translation vector $\mathbf{t}$. The position at time step t ($t = 0, \dots, 8$) can then be computed by applying P^t and $\mathbf{t} \cdot t$ to an initial coordinate. The discretized velocities are calculated by taking differences between the consecutive time steps. We further add slight deformations to the structures by adding small Gaussian noise.

Objective and loss function. After constructing a dataset of N_{train} such trajectories, we predict, given four input time steps, the next four

input timesteps and compare them against the ground-truth trajectories for all objects. We define a loss function

$$\mathcal{L}_{\text{MSE}} := \frac{1}{N_{\text{locations}}} \sum_{t=1}^{N_t} \sum_{y=1}^{N_{\text{locations}}} \sum_{p=1}^{N_p} (x_{t,y,p} - \hat{x}_{t,y,p})^2$$

that expresses for all train datapoints and for all time steps the discrepancy between the predicted three-dimensional locations and the ground-truth ones. When we predict positions, $N_p = 3$. When we also predict the velocities, we include those as well (i.e., then p sums to 6). In our experiments we have $N_t = 4$ predicted output time steps and $N_{\text{locations}} = 32$. We average this loss over 1024 validation and test trajectories.

GCAN implementation. In this experiment, we consider the geometric algebra $G_{3,0,1}$. The highest-order proper isometry is a screw motion (a simultaneous rotation and translation), which is implemented using the components

$$a := a_0 1 + a_{01} e_{01} + a_{02} e_{02} + a_{03} e_{03} + a_{12} e_{12} + a_{13} e_{13} + a_{23} e_{23} + a_{0123} e_{0123},$$

where the coefficients are free parameters to be optimized. Since points in geometric algebra are defined by intersection of three planes. As such, we encode the x , y , z -coordinates of a point as $x := x e_{012} + y e_{013} + z e_{023}$ and transform them using group actions. The e_{123} (the dual of e_0) parameterizes a distance from the origin, as such we get $x := x e_{012} + y e_{013} + z e_{023} + \delta e_{123}$, where we leave δ in each layer as a free parameter. The full input representation to the GCA-MLP has $N_{\text{locations}} \cdot N_t$ channels. In contrast: a naive MLP has $N_{\text{locations}} \cdot N_p$ input channels. The GCA-MLP then transforms this input using geometric algebra linear layers.

The GCA-GNN additionally encodes the positions as graph nodes and therefore only has N_t input channels. Each message and each node update network of the message passing layers is implemented as a GCA-MLP as described above.

When we include the velocities then $\mathbf{x} := v_x e_1 + v_y e_2 + v_z e_3 + x e_{012} + y e_{013} + z e_{023}$. Importantly, we can use the same weights a to transform this multivector. However, to incorporate the knowledge of the additional velocities into the neural network, we need to transform this multivector.

6 Conclusion

By understanding the fundamentals of geometric algebra and Clifford algebras, we gain powerful mathematical tools for representing and manipulating geometric transformations. These tools are particularly valuable in the context of neural networks that aim to model physical systems and processes involving complex spatial relationships, as they enable the networks to inherently respect and preserve the underlying geometric structures of the data.

Список литературы

- Ruhe, David и др. (2023). «Geometric clifford algebra networks». В: *International Conference on Machine Learning*. PMLR, с. 29306—29337.
- Trautman, Andrzej (2006). «Clifford algebras and their representations». В: *Encyclopedia of Mathematical Physics* 1.2, с. 518—530.